

1. Python Program to Print Hello world!
2. Python Program to Add Two Numbers
3. Python Program to Find the Square Root
4. Python Program to Calculate the Area of a Triangle
5. Python Program to Solve Quadratic Equation
6. Python Program to Swap Two Variables
7. Python Program to Generate a Random Number
8. Python Program to Convert Kilometers to Miles
9. Python Program to Convert Celsius To Fahrenheit
10. Python Program to Check if a Number is Positive, Negative or 0
11. Python Program to Check if a Number is Odd or Even
12. Python Program to Check Prime Number
13. Python Program to Find the Factorial of a Number
14. Python Program to Display the multiplication Table
15. Python Program to Print the Fibonacci sequence
16. Python Program to Check Armstrong Number
17. Python Program to Find the Sum of Natural Numbers
18. Python Program to Convert Decimal to Binary
19. Python Program to Convert Octal to Hexadecimal
20. Python Program to Find ASCII Value of Character
21. Python Program to Find HCF or GCD
22. Python Program to Find LCM
23. Python Program to Find the Factors of a Number
24. Python Program to Make a Simple Calculator
25. Python Program to Display Fibonacci Sequence Using Recursion
26. Python Program to Find Sum of Natural Numbers Using Recursion
27. Python Program to Find Factorial of Number Using Recursion
28. Python Program to Convert Decimal to Binary Using Recursion
29. Python Program to Add Two Matrices
30. Python Program to Transpose a Matrix
31. Python Program to Multiply Two Matrices
32. Python Program to Check Whether a String is Palindrome or Not
33. Python Program to Remove Punctuations From a String
34. Python Program to Sort Words in Alphabetic Order
35. Python Program to Illustrate Different Set Operations
36. Python Program to Count the Number of Each Vowel
37. Python Program to Create Pyramid Patterns
38. Python Program to Merge Two Dictionaries
39. Python Program to Slice Lists
40. Python Program to Iterate Over Dictionaries Using for Loop
41. Python Program to Sort a Dictionary by Value
42. Python Program to Check If a List is Empty
43. Python Program to Copy a File
44. Python Program to Concatenate Two Lists
45. Python Program to Check if a Key is Already Present in a Dictionary
46. Python Program to Get the Last Element of the List
47. Python Program to Get a Substring of a String
48. Python Program Read a File Line by Line Into a List
49. Python Program to Randomly Select an Element From the List
50. Python Program to Count the Occurrence of an Item in a List
51. Python Program to Get Line Count of a File
52. Python Program to Reverse a Number
53. Python Program to Compute the Power of a Number
54. Python Program to Count the Number of Digits Present In a Number
55. Python Program to Check If Two Strings are Anagram
56. Python Program to Capitalize the First Character of a String
57. Python Program to Count the Number of Occurrence of a Character in String
58. Python Program to Remove Duplicate Element From a List
59. Python Program to Find the Largest Element in an Array
60. Python Program to Find the Second Largest Element in an Array

- | | | |
|--|--|--|
| <ul style="list-style-type: none">61. Python Program to Count the Number of Words in a String62. Python Program to Check Whether a Number is a Perfect Number or Not63. Python Program to Find the Sum of Digits in a Number64. Python Program to Find the Intersection of Two Lists65. Python Program to Calculate the Power Set of a Set66. Python Program to Find the Union of Two Sets67. Python Program to Find the Mean, Median, and Mode of a List of Numbers68. Python Program to Find the Smallest Element in an Array69. Python Program to Implement Selection Sort Algorithm70. Python Program to Implement Bubble Sort Algorithm71. Python Program to Implement Insertion Sort Algorithm72. Python Program to Implement Merge Sort Algorithm73. Python Program to Implement Quick Sort Algorithm74. Python Program to Implement Binary Search Algorithm75. Python Program to Convert a List of Tuples into a Dictionary76. Python Program to Convert a Dictionary into a List of Tuples | <ul style="list-style-type: none">77. Python Program to Find the Cube of a Number78. Python Program to Find the Sum of Cubes of Natural Numbers79. Python Program to Calculate Simple Interest80. Python Program to Calculate Compound Interest81. Python Program to Find the Area of a Circle82. Python Program to Find the Circumference of a Circle83. Python Program to Check Whether a Year is a Leap Year or Not84. Python Program to Find the LCM of Two Numbers using GCD85. Python Program to Calculate the Area of a Rectangle86. Python Program to Calculate the Perimeter of a Rectangle87. Python Program to Find the Smallest Among Three Numbers using Conditional Operator88. Python Program to Find the Nth Fibonacci Number89. Python Program to Find the Sum of Series: $1 + 1/2 + 1/3 + \dots + 1/n$90. Python Program to Check Whether a String is a Pangram or Not91. Python Program to Find the Length of a String92. Python Program to Find the Maximum Occurring Character in a String93. Python Program to Find the Reverse of a String | <ul style="list-style-type: none">94. Python Program to Find the Factorial of a Large Number using Libraries95. Python Program to Find the Number of Digits in a Number using Recursion96. Python Program to Find the Average of Numbers in a List97. Python Program to Find the Prime Factors of a Number98. Python Program to Find the Intersection of Two Sets99. Python program to reverse a number100. Python program to generate a random number |
|--|--|--|

SOLUTIONS

Note: Don't solely depend on these solutions, please try the programs with your own solutions also!

1.

```
print("Hello World!")
```

2.

```
num1 = 10
```

```
num2 = 20
```

```
sum = num1 + num2
```

```
print("Sum:", sum)
```

3.

```
import math
```

```
num = 16
```

```
square_root = math.sqrt(num)
```

```
print("Square root:", square_root)
```

4.

```
base = 10
```

```
height = 5
```

```
area = 0.5 * base * height
```

```
print("Area of the triangle:", area)
```

5.

```
import cmath
```

```
a = 1
```

```
b = 5
```

```
c = 6
```

```
d = (b ** 2) - (4 * a * c)
```

```
sol1 = (-b - cmath.sqrt(d)) / (2 * a)
```

```
sol2 = (-b + cmath.sqrt(d)) / (2 * a)
```

```
print("Solutions:", sol1, sol2)
```

6.

```
x = 5
```

```
y = 10
```

```
temp = x
```

```
x = y
```

```
y = temp
```

```
print("x after swapping:", x)
```

```
print("y after swapping:", y)
```

7.

```
import random
```

```
random_number = random.randint(1, 100)
```

```
print("Random number:", random_number)
```

8.

```
kilometers = 5
```

```
conv_factor = 0.621371
```

```
miles = kilometers * conv_factor
```

```
print("Miles:", miles)
```

9.

```
celsius = 37
```

```
fahrenheit = (celsius * 9/5) + 32
```

```
print("Temperature in Fahrenheit:", fahrenheit)
```

10.

```
num = 10
```

```
if num > 0:
```

```
    print("Positive number")
```

```
elif num == 0:
```

```
    print("Zero")
```

```
else:
```

```
    print("Negative number")
```

11.

```
num = 7
```

```
if num % 2 == 0:
```

```
    print("Even number")
```

```
else:
```

```
    print("Odd number")
```

12.

```
num = 17
```

```
if num > 1:
```

```
    for i in range(2, int(num/2)+1):
```

```
        if (num % i) == 0:
```

```
            print(num, "is not a prime number")
```

```
            break
```

```
else:
```

```
    print(num, "is a prime number")
```

```
else:
```

```
    print(num, "is not a prime number")
```

13.

```
num = 5
```

```
factorial = 1
```

```
if num < 0:
```

```
    print("Factorial does not exist for negative numbers")
```

```
elif num == 0:
```

```
    print("Factorial of 0 is 1")
```

```
else:
```

```
    for i in range(1, num + 1):
```

```
factorial *= i
```

```
print("Factorial of", num, "is", factorial)
```

14.

```
num = 9
```

```
for i in range(1, 11):
```

```
    print(num, 'x', i, '=', num*i)
```

15.

```
nterms = 10
```

```
n1, n2 = 0, 1
```

```
count = 0
```

```
if nterms <= 0:
```

```
    print("Please enter a positive integer")
```

```
elif nterms == 1:
```

```
    print("Fibonacci sequence upto", nterms, ":")
```

```
    print(n1)
```

```
else:
```

```
    print("Fibonacci sequence:")
```

```
    while count < nterms:
```

```
        print(n1)
```

```
        nth = n1 + n2
```

```
        n1 = n2
```

```
        n2 = nth
```

```
        count += 1
```

16.

```
num = 153
```

```
order = len(str(num))
```

```
sum = 0
```

```
temp = num
```

```
while temp > 0:
```

```
    digit = temp % 10
```

```
    sum += digit ** order
```

```
    temp //= 10
```

```
if num == sum:
```

```
    print(num, "is an Armstrong number")
```

```
else:
```

```
    print(num, "is not an Armstrong number")
```

```
17.
```

```
num = 10
```

```
sum = 0
```

```
if num < 0:
```

```
    print("Enter a positive number")
```

```
else:
```

```
    for i in range(1, num + 1):
```

```
        sum += i
```

```
    print("Sum:", sum)
```

```
18.
```

```
decimal = 10
```

```
binary = bin(decimal)
```

```
print("Binary representation:", binary)
```

```
19.
```

```
octal = '17'
```

```
hexadecimal = hex(int(octal, 8))
```

```
print("Hexadecimal representation:", hexadecimal)
```

```
20.
```

```
char = 'A'
ascii_value = ord(char)
print("ASCII value of", char, "is", ascii_value)
```

21.

```
def find_hcf(x, y):
    while(y):
        x, y = y, x % y
    return x
```

```
num1 = 24
```

```
num2 = 36
```

```
print("The HCF of", num1, "and", num2, "is", find_hcf(num1, num2))
```

22.

```
def find_lcm(x, y):
    lcm = (x*y)//find_hcf(x, y)
    return lcm
```

```
num1 = 24
```

```
num2 = 36
```

```
print("The LCM of", num1, "and", num2, "is", find_lcm(num1, num2))
```

23.

```
def find_factors(num):
    factors = []
    for i in range(1, num+1):
        if num % i == 0:
            factors.append(i)
    return factors
```

```
number = 24
print("Factors of", number, "are:", find_factors(number))
```

24.

```
def add(x, y):
    return x + y
```

```
def subtract(x, y):
    return x - y
```

```
def multiply(x, y):
    return x * y
```

```
def divide(x, y):
    return x / y
```

```
num1 = 10
num2 = 5
```

```
print("Addition:", add(num1, num2))
print("Subtraction:", subtract(num1, num2))
print("Multiplication:", multiply(num1, num2))
print("Division:", divide(num1, num2))
```

25.

```
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2)
```

```
nterms = 10
```

```
if nterms <= 0:
```

```
    print("Please enter a positive integer")
```

```
else:
```

```
    print("Fibonacci sequence:")
```

```
    for i in range(nterms):
```

```
        print(fibonacci(i))
```

26.

```
def sum_of_natural_numbers(n):
```

```
    if n <= 1:
```

```
        return n
```

```
    else:
```

```
        return n + sum_of_natural_numbers(n-1)
```

```
num = 10
```

```
print("Sum of first", num, "natural numbers is", sum_of_natural_numbers(num))
```

27.

```
def factorial(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    else:
```

```
        return n * factorial(n-1)
```

```
num = 5
```

```
print("Factorial of", num, "is", factorial(num))
```

28.

```
def decimal_to_binary(n):
```

```
if n > 1:
    decimal_to_binary(n//2)
print(n % 2, end="")
```

```
decimal = 10
print("Binary representation:", end=' ')
decimal_to_binary(decimal)
```

29.

```
X = [[1,2,3],
      [4,5,6],
      [7,8,9]]
```

```
Y = [[9,8,7],
      [6,5,4],
      [3,2,1]]
```

```
result = [[0,0,0],
           [0,0,0],
           [0,0,0]]
```

```
for i in range(len(X)):
    for j in range(len(X[0])):
        result[i][j] = X[i][j] + Y[i][j]
```

```
for r in result:
    print(r)
```

30.

```
X = [[1,2],
      [4,5],
```

[7,8]]

```
result = [[0,0,0],  
          [0,0,0]]
```

```
for i in range(len(X)):  
    for j in range(len(X[0])):  
        result[j][i] = X[i][j]
```

```
for r in result:  
    print(r)
```

31.

```
X = [[1,2,3],  
     [4,5,6],  
     [7,8,9]]
```

```
Y = [[9,8,7],  
     [6,5,4],  
     [3,2,1]]
```

```
result = [[0,0,0],  
          [0,0,0],  
          [0,0,0]]
```

```
for i in range(len(X)):  
    for j in range(len(Y[0])):  
        for k in range(len(Y)):  
            result[i][j] += X[i][k] * Y[k][j]
```

for r in result:

print(r)

32.

def is_palindrome(s):

s = s.lower()

return s == s[::-1]

string = "radar"

if is_palindrome(string):

print("Palindrome")

else:

print("Not a Palindrome")

33.

import string

def remove_punctuation(s):

return s.translate(str.maketrans("", "", string.punctuation))

string = "Hello, World!"

print("String without punctuations:", remove_punctuation(string))

34.

def sort_words(s):

words = s.split()

words.sort()

return ' '.join(words)

sentence = "Python Program to Sort Words in Alphabetic Order"

print("Sorted words:", sort_words(sentence))

35.

```
set1 = {1, 2, 3, 4, 5}
```

```
set2 = {4, 5, 6, 7, 8}
```

```
print("Union:", set1 | set2)
```

```
print("Intersection:", set1 & set2)
```

```
print("Difference:", set1 - set2)
```

```
print("Symmetric Difference:", set1 ^ set2)
```

36.

```
def count_vowels(s):
```

```
    vowels = 'aeiou'
```

```
    count = {}.fromkeys(vowels, 0)
```

```
    for char in s:
```

```
        if char in count:
```

```
            count[char] += 1
```

```
    return count
```

```
string = "Python Program to Count the Number of Each Vowel"
```

```
print("Count of vowels:", count_vowels(string.lower()))
```

37.

```
def print_pyramid(rows):
```

```
    for i in range(0, rows):
```

```
        for j in range(0, i + 1):
```

```
            print("*", end=' ')
```

```
        print("\n")
```

```
rows = 5
```

```
print_pyramid(rows)
```

38.

```
def merge_dicts(dict1, dict2):
```

```
return {**dict1, **dict2}
```

```
dict1 = {'a': 1, 'b': 2}
```

```
dict2 = {'c': 3, 'd': 4}
```

```
merged_dict = merge_dicts(dict1, dict2)
```

```
print("Merged dictionary:", merged_dict)
```

39.

```
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# Slice from index 2 to 5
```

```
slice_result = my_list[2:6]
```

```
print("Sliced list:", slice_result)
```

40.

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
```

```
for key, value in my_dict.items():
```

```
    print(key, "->", value)
```

41.

```
my_dict = {'b': 4, 'a': 3, 'c': 1}
```

```
sorted_dict = dict(sorted(my_dict.items(), key=lambda item: item[1]))
```

```
print("Sorted dictionary by value:", sorted_dict)
```

42.

```
my_list = []
```

```
if not my_list:
```

```
    print("List is empty")
```

```
else:
```

```
print("List is not empty")
```

43.

```
import shutil
```

```
shutil.copy("source_file.txt", "destination_file.txt")
```

```
print("File copied successfully.")
```

44.

```
list1 = [1, 2, 3]
```

```
list2 = [4, 5, 6]
```

```
concatenated_list = list1 + list2
```

```
print("Concatenated list:", concatenated_list)
```

45.

```
my_dict = {'a': 1, 'b': 2}
```

```
key_to_check = 'a'
```

```
if key_to_check in my_dict:
```

```
    print(key_to_check, "is present in the dictionary")
```

```
else:
```

```
    print(key_to_check, "is not present in the dictionary")
```

46.

```
my_list = [1, 2, 3, 4, 5]
```

```
last_element = my_list[-1]
```

```
print("Last element of the list:", last_element)
```

47.

```
my_string = "Hello, World!"
```

```
substring = my_string[7:]  
print("Substring:", substring)
```

48.

```
with open("file.txt", "r") as file:  
    lines = file.readlines()  
    lines = [line.strip() for line in lines]
```

```
print("Lines read from file:", lines)
```

49.

```
import random
```

```
my_list = [1, 2, 3, 4, 5]
```

```
random_element = random.choice(my_list)  
print("Randomly selected element:", random_element)
```

50.

```
my_list = [1, 2, 3, 4, 1, 2, 1, 1]
```

```
count_of_ones = my_list.count(1)  
print("Count of 1s in the list:", count_of_ones)
```

51.

```
with open("file.txt", "r") as file:  
    line_count = sum(1 for line in file)
```

```
print("Line count of the file:", line_count)
```

52.

```
num = 12345
```

```
reversed_num = int(str(num)[::-1])
```

```
print("Reversed number:", reversed_num)
```

53.

```
base = 2
```

```
exponent = 3
```

```
result = base ** exponent
```

```
print("Result:", result)
```

54.

```
num = 12345
```

```
num_of_digits = len(str(num))
```

```
print("Number of digits in", num, ":", num_of_digits)
```

55.

```
def are_anagrams(str1, str2):
```

```
    return sorted(str1) == sorted(str2)
```

```
string1 = "listen"
```

```
string2 = "silent"
```

```
if are_anagrams(string1, string2):
```

```
    print("Strings are anagrams")
```

```
else:
```

```
    print("Strings are not anagrams")
```

56.

```
def capitalize_first_char(s):
```

```
    return s.capitalize()
```

```
string = "hello world"
```

```
print("String with first character capitalized:", capitalize_first_char(string))
```

57.

```
def count_occurrence(string, char):  
    return string.count(char)
```

```
string = "hello world"
```

```
char_to_count = 'o'
```

```
print("Number of occurrences of '{}' in '{}' is: {}".format(char_to_count, string, count_occurrence(string, char_to_count)))
```

58.

```
def remove_duplicates(lst):  
    return list(set(lst))
```

```
my_list = [1, 2, 3, 2, 4, 3, 5]
```

```
print("List after removing duplicates:", remove_duplicates(my_list))
```

59.

```
def find_largest(arr):  
    return max(arr)
```

```
my_array = [10, 20, 30, 40, 50]
```

```
print("Largest element in the array:", find_largest(my_array))
```

60.

```
def find_second_largest(arr):  
    sorted_arr = sorted(arr)  
    return sorted_arr[-2]
```

```
my_array = [10, 20, 30, 40, 50]
```

```
print("Second largest element in the array:", find_second_largest(my_array))
```

61.

```
def count_words(string):
```

```
return len(string.split())
```

```
string = "Python is awesome"
```

```
print("Number of words in the string:", count_words(string))
```

62.

```
def is_perfect_number(num):
```

```
    sum_of_divisors = sum([i for i in range(1, num) if num % i == 0])
```

```
    return sum_of_divisors == num
```

```
number = 28
```

```
if is_perfect_number(number):
```

```
    print(number, "is a perfect number")
```

```
else:
```

```
    print(number, "is not a perfect number")
```

63.

```
def sum_of_digits(num):
```

```
    return sum(int(digit) for digit in str(num))
```

```
number = 12345
```

```
print("Sum of digits in", number, "is:", sum_of_digits(number))
```

64.

```
def intersection(lst1, lst2):
```

```
    return list(set(lst1) & set(lst2))
```

```
list1 = [1, 2, 3, 4, 5]
```

```
list2 = [4, 5, 6, 7, 8]
```

```
print("Intersection of lists:", intersection(list1, list2))
```

65.

```
from itertools import chain, combinations
```

```
def powerset(s):  
    return list(chain.from_iterable(combinations(s, r) for r in range(len(s)+1)))
```

```
my_set = {1, 2, 3}  
print("Power set of the set:", powerset(my_set))
```

66.

```
def union(set1, set2):  
    return set1.union(set2)
```

```
set1 = {1, 2, 3}  
set2 = {3, 4, 5}  
print("Union of sets:", union(set1, set2))
```

67.

```
from statistics import mean, median, mode
```

```
my_list = [1, 2, 2, 3, 4, 4, 4, 5]
```

```
mean_value = mean(my_list)  
median_value = median(my_list)  
mode_value = mode(my_list)
```

```
print("Mean:", mean_value)  
print("Median:", median_value)  
print("Mode:", mode_value)
```

68.

```
def find_smallest(arr):  
    return min(arr)
```

```
my_array = [10, 20, 30, 40, 50]
print("Smallest element in the array:", find_smallest(my_array))
```

69.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

```
my_array = [64, 25, 12, 22, 11]
selection_sort(my_array)
print("Sorted array using Selection Sort:", my_array)
```

70.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n-1):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

```
my_array = [64, 25, 12, 22, 11]
bubble_sort(my_array)
print("Sorted array using Bubble Sort:", my_array)
```

71.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
```

```
key = arr[i]
j = i - 1
while j >= 0 and key < arr[j]:
    arr[j + 1] = arr[j]
    j -= 1
arr[j + 1] = key
```

```
my_array = [64, 25, 12, 22, 11]
insertion_sort(my_array)
print("Sorted array using Insertion Sort:", my_array)
```

72.

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        L = arr[:mid]
        R = arr[mid:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0

        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
```

```
k += 1
```

```
while i < len(L):
```

```
    arr[k] = L[i]
```

```
    i += 1
```

```
    k += 1
```

```
while j < len(R):
```

```
    arr[k] = R[j]
```

```
    j += 1
```

```
    k += 1
```

```
my_array = [64, 25, 12, 22, 11]
```

```
merge_sort(my_array)
```

```
print("Sorted array using Merge Sort:", my_array)
```

73.

```
def quick_sort(arr):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    pivot = arr[len(arr) // 2]
```

```
    left = [x for x in arr if x < pivot]
```

```
    middle = [x for x in arr if x == pivot]
```

```
    right = [x for x in arr if x > pivot]
```

```
    return quick_sort(left) + middle + quick_sort(right)
```

```
my_array = [64, 25, 12, 22, 11]
```

```
my_array = quick_sort(my_array)
```

```
print("Sorted array using Quick Sort:", my_array)
```

74.

```
def binary_search(arr, x):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] < x:
```

```
            low = mid + 1
```

```
        elif arr[mid] > x:
```

```
            high = mid - 1
```

```
        else:
```

```
            return mid
```

```
    return -1
```

```
my_array = [2, 3, 4, 10, 40]
```

```
x = 10
```

```
result = binary_search(my_array, x)
```

```
if result != -1:
```

```
    print("Element", x, "is present at index", result)
```

```
else:
```

```
    print("Element", x, "is not present in array")
```

75.

```
def convert_to_dict(lst):
```

```
    return dict(lst)
```

```
my_list_of_tuples = [("a", 1), ("b", 2), ("c", 3)]
```

```
my_dict = convert_to_dict(my_list_of_tuples)
```

```
print("Dictionary:", my_dict)
```

76.

```
def convert_to_list_of_tuples(my_dict):
```

```
return list(my_dict.items())
```

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
```

```
my_list_of_tuples = convert_to_list_of_tuples(my_dict)
```

```
print("List of tuples:", my_list_of_tuples)
```

77.

```
def cube(num):
```

```
    return num ** 3
```

```
number = 5
```

```
print("Cube of", number, "is:", cube(number))
```

78.

```
def sum_of_cubes(n):
```

```
    return sum(i**3 for i in range(1, n+1))
```

```
n = 3
```

```
print("Sum of cubes of first", n, "natural numbers is:", sum_of_cubes(n))
```

79.

```
def simple_interest(principal, rate, time):
```

```
    return (principal * rate * time) / 100
```

```
principal = 1000
```

```
rate = 5
```

```
time = 2
```

```
print("Simple Interest:", simple_interest(principal, rate, time))
```

80.

```
def compound_interest(principal, rate, time):
```

```
    return principal * ((1 + rate / 100) ** time - 1)
```

```
principal = 1000
```

```
rate = 5
```

```
time = 2
```

```
print("Compound Interest:", compound_interest(principal, rate, time))
```

81.

```
import math
```

```
def area_of_circle(radius):
```

```
    return math.pi * radius**2
```

```
radius = 5
```

```
print("Area of the circle with radius", radius, "is:", area_of_circle(radius))
```

82.

```
import math
```

```
def circumference_of_circle(radius):
```

```
    return 2 * math.pi * radius
```

```
radius = 5
```

```
print("Circumference of the circle with radius", radius, "is:", circumference_of_circle(radius))
```

83.

```
def is_leap_year(year):
```

```
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
```

```
        return True
```

```
    else:
```

```
        return False
```

```
year = 2024
```

```
if is_leap_year(year):
```

```
print(year, "is a leap year")
```

```
else:
```

```
print(year, "is not a leap year")
```

84.

```
def gcd(x, y):
```

```
    while y:
```

```
        x, y = y, x % y
```

```
    return x
```

```
def lcm(x, y):
```

```
    return (x * y) // gcd(x, y)
```

```
num1 = 12
```

```
num2 = 15
```

```
print("LCM of", num1, "and", num2, "is:", lcm(num1, num2))
```

85.

```
def area_of_rectangle(length, width):
```

```
    return length * width
```

```
length = 5
```

```
width = 4
```

```
print("Area of the rectangle with length", length, "and width", width, "is:", area_of_rectangle(length, width))
```

86.

```
def perimeter_of_rectangle(length, width):
```

```
    return 2 * (length + width)
```

```
length = 5
```

```
width = 4
```

```
print("Perimeter of the rectangle with length", length, "and width", width, "is:",  
      perimeter_of_rectangle(length, width))
```

87.

```
def find_smallest(num1, num2, num3):  
    return min(num1, num2, num3)
```

```
num1 = 10
```

```
num2 = 5
```

```
num3 = 7
```

```
print("Smallest number among", num1, ",", num2, ", and", num3, "is:", find_smallest(num1, num2, num3))
```

88.

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)
```

```
n = 10
```

```
print("The", n, "th Fibonacci number is:", fibonacci(n))
```

89.

```
def sum_of_series(n):  
    return sum(1/i for i in range(1, n+1))
```

```
n = 5
```

```
print("Sum of the series 1 + 1/2 + 1/3 + ... + 1/{ } is:".format(n), sum_of_series(n))
```

90.

```
import string
```

```
def is_pangram(s):
```

```
alphabet = set(string.ascii_lowercase)
return set(s.lower()) >= alphabet
```

```
string_to_check = "The quick brown fox jumps over the lazy dog"
```

```
if is_pangram(string_to_check):
```

```
    print("The string is a pangram")
```

```
else:
```

```
    print("The string is not a pangram")
```

```
91.
```

```
def string_length(s):
```

```
    return len(s)
```

```
string = "Hello, World!"
```

```
print("Length of the string is:", string_length(string))
```

```
92.
```

```
from collections import Counter
```

```
def max_occurrence(s):
```

```
    counter = Counter(s)
```

```
    max_count = max(counter.values())
```

```
    max_chars = [char for char, count in counter.items() if count == max_count]
```

```
    return max_chars
```

```
string = "Hello, World!"
```

```
print("Maximum occurring character(s) in the string:", max_occurrence(string))
```

```
93
```

```
def reverse_string(s):
```

```
    return s[::-1]
```

```
string = "Hello, World!"  
print("Reverse of the string is:", reverse_string(string))
```

94.

```
import math
```

```
def factorial(n):  
    return math.factorial(n)
```

```
number = 10  
print("Factorial of", number, "is:", factorial(number))
```

95.

```
def count_digits(n):  
    if n == 0:  
        return 0  
    return 1 + count_digits(n // 10)
```

```
number = 12345  
print("Number of digits in", number, "is:", count_digits(number))
```

96.

```
def average(lst):  
    return sum(lst) / len(lst)
```

```
my_list = [1, 2, 3, 4, 5]  
print("Average of the numbers in the list is:", average(my_list))
```

97.

```
def prime_factors(n):  
    factors = []  
    divisor = 2  
    while n > 1:
```

```
while n % divisor == 0:
    factors.append(divisor)
    n //= divisor
divisor += 1
return factors
```

```
number = 60
print("Prime factors of", number, "are:", prime_factors(number))
```

98.

```
def intersection(set1, set2):
    return set1.intersection(set2)
```

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print("Intersection of sets:", intersection(set1, set2))
```

99.

```
def reverse_number(n):
    rev = 0
    while n > 0:
        rev = rev * 10 + n % 10
        n //= 10
    return rev
```

```
number = 12345
print("Reverse of", number, "is:", reverse_number(number))
```

100.

```
import random
```

```
def generate_random_number(start, end):
```

```
return random.randint(start, end)
```

```
start = 1
```

```
end = 100
```

```
print("Random number between", start, "and", end, "is:", generate_random_number(start, end))
```

IG/NITHESH.TECH